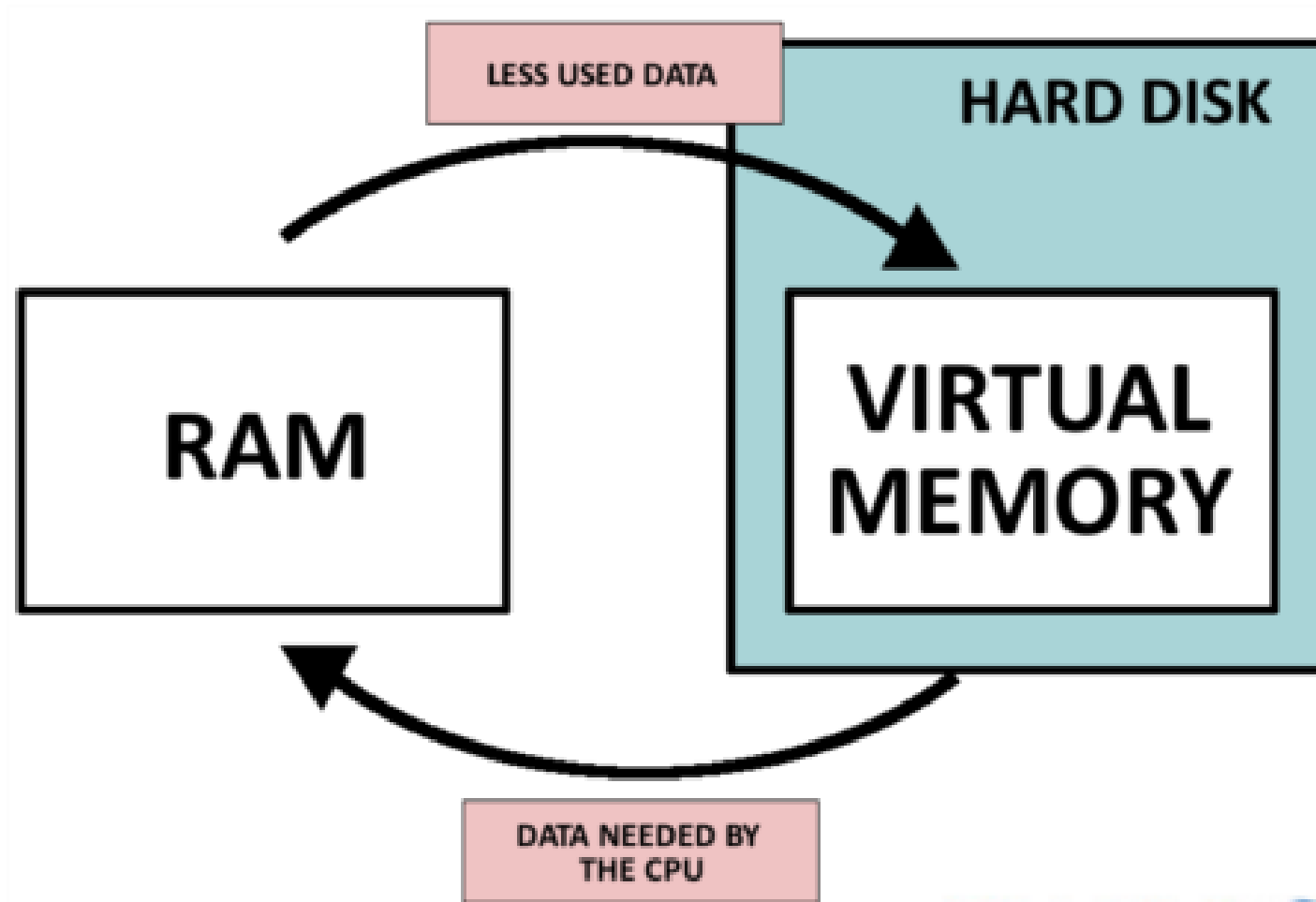


Unit-5

Memory Management – Part B

Virtual Memory



Virtual Memory (contd..)

- In multiprogramming system, when we use multiple programs, the space of primary memory RAM may not be enough for storing all the programs.
- In this case, OS use free of secondary memory like hard disk to store those contents. This space in secondary memory used for storing the contents of primary memory is called as **virtual memory**.
- It is used for two purposes:
 - It allows us to extend the use of physical memory by using disk.
 - It allows us to have memory protection, because each virtual address is translated to a physical address.

Limitations of Virtual memory

- It is slower. It's generally better to have as much physical memory as possible so programs work directly from RAM or physical memory.
- The use of virtual memory slows a computer because data must be mapped between virtual and physical memory, which requires extra hardware support for address translation.

Logical versus Physical Address space (revisited)

- The memory address is used to identify or locate a particular memory location in the computer.
- Memory addresses are of two types:
 - 1) Logical Address
 - 2) Physical Address

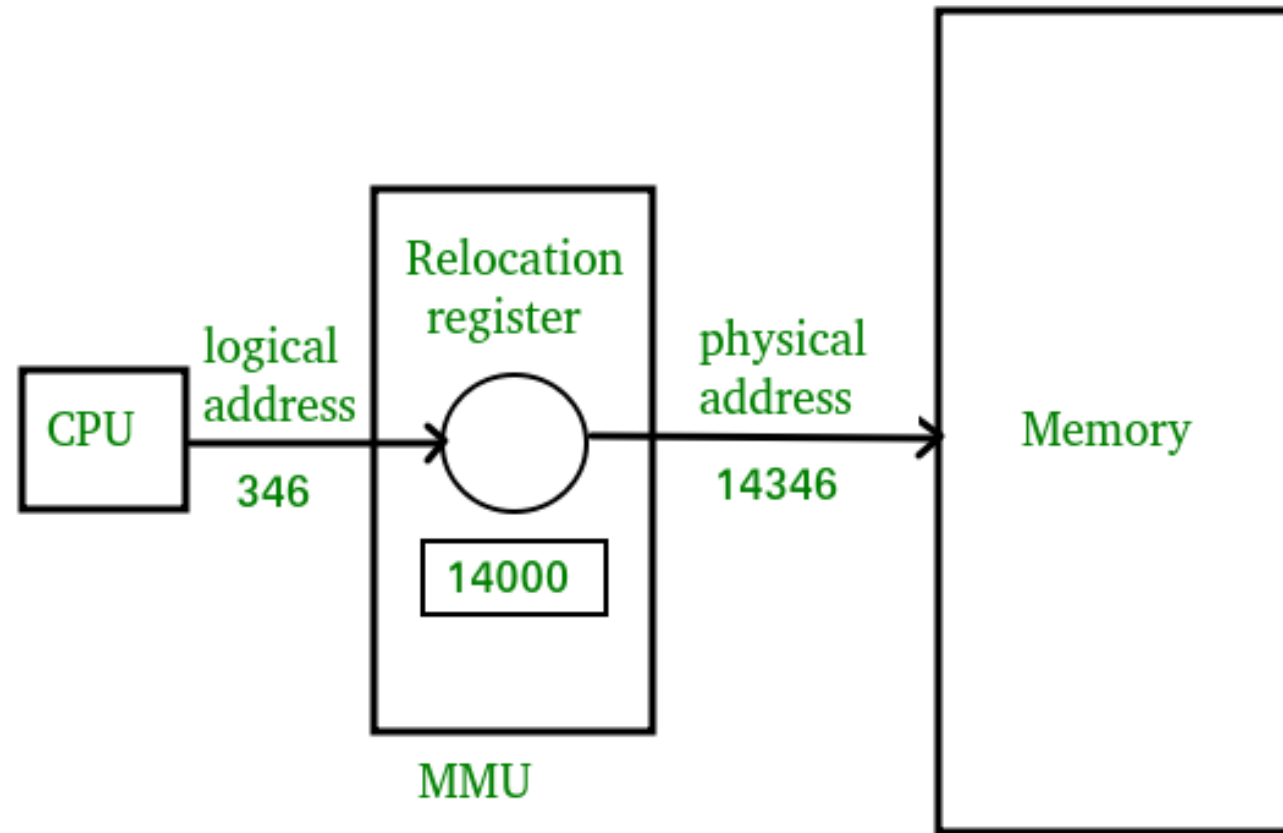
Logical Address

- It is the address generated by CPU while a program is running.
- It is virtual as it does not exist physically.
- It is used as a reference to access the physical memory location.
- The logical address is mapped to its corresponding physical address by a hardware device called Memory-Management Unit (MMU).
- The address-binding methods used by MMU generates identical logical and physical address during compile and load time but generates different logical and physical address during run-time.

Physical Address

- It identifies a physical location in a memory.
- MMU (Memory-Management Unit) computes the physical address for the corresponding logical address.
- The user never deals with the physical address.
- Instead, the physical address is accessed by its corresponding logical address by the user.
- The user program generates the logical address, but the program needs physical memory for its execution. Hence, the logical address must be mapped to the physical address before they are used.

Logical to physical address mapping



Logical versus Physical Address

Logical Address	Physical Address
It is generated by CPU in perspective of a program	It is a location that exists in the memory unit.
The set of all logical addresses generated by CPU for a program is called logical address space.	The set of all physical address mapped to corresponding logical address is referred as physical address space.
It is called as virtual address as the address doesn't exists physically in the memory unit.	The physical address is a location in the memory unit that can be accessed physically.
The logical address is generated by the CPU while program is running.	The physical address is computed by the MMU (Memory Management Unit)

Paging

- In OS, paging is a memory management scheme by which a computer stores and retrieves data from secondary storage for use in main memory.
- In this scheme, the operating system retrieves data from secondary storage in same-size blocks called pages.
- Paging is an important part of virtual memory implementations in modern OS.
- The main idea behind the paging is to divide each process in the form of pages. The main memory will also be divided in the form of frames.

Paging (contd..)

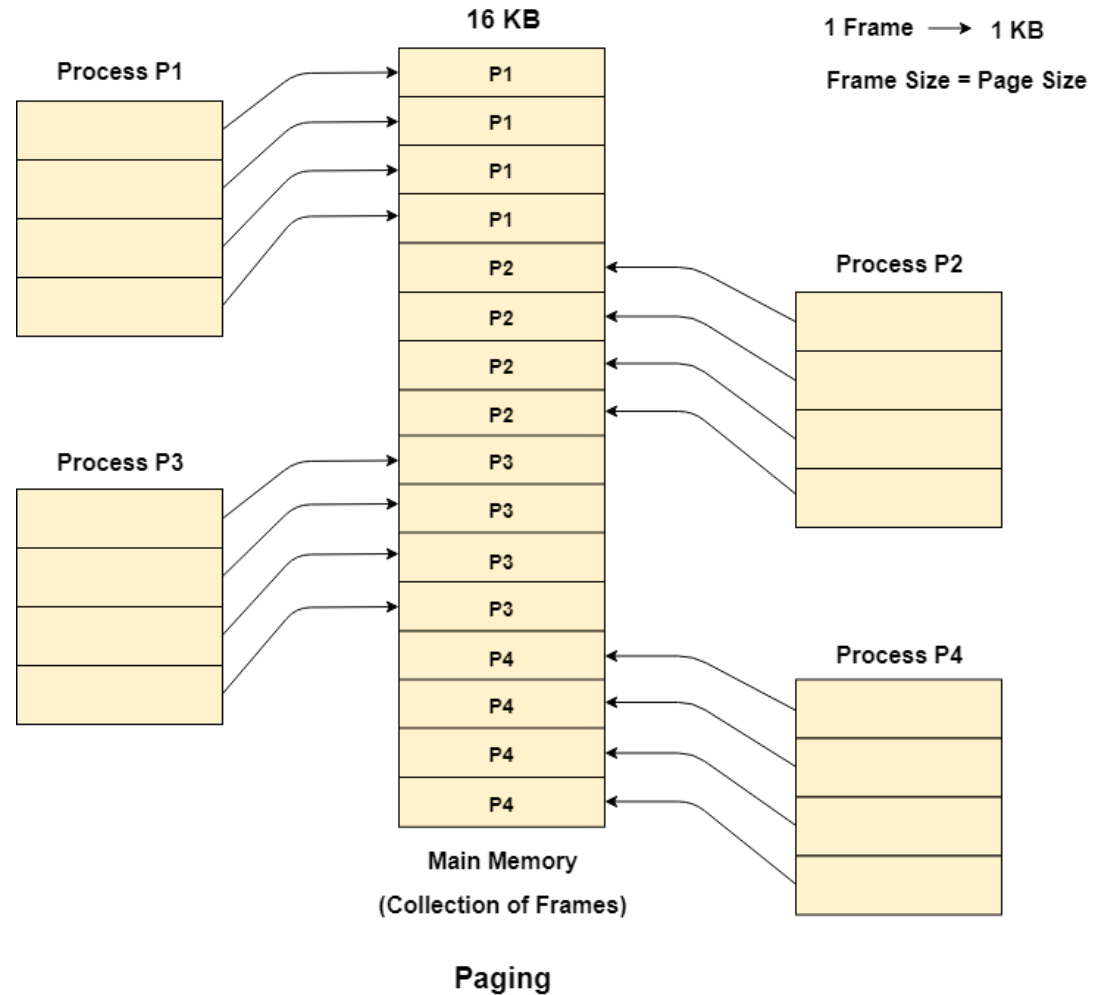
- One page of the process is to be stored in one of the frames of the memory. The pages can be stored at the different locations of the memory but the priority is always to find the contiguous frames or holes.
- Pages of the process are brought into the main memory only when they are required otherwise they reside in the secondary storage.
- Different operating system defines different frame sizes. The sizes of each frame must be equal. Considering the fact that the pages are mapped to the frames in Paging, page size needs to be as same as frame size.

Paging (contd..)

Example

- Let us consider the main memory size 16 Kb and Frame size is 1 KB therefore the main memory will be divided into the collection of 16 frames of 1 KB each.
- There are 4 processes in the system that is P1, P2, P3 and P4 of 4 KB each. Each process is divided into pages of 1 KB each so that one page can be stored in one frame.
- Initially, all the frames are empty therefore pages of the processes will get stored in the contiguous way.

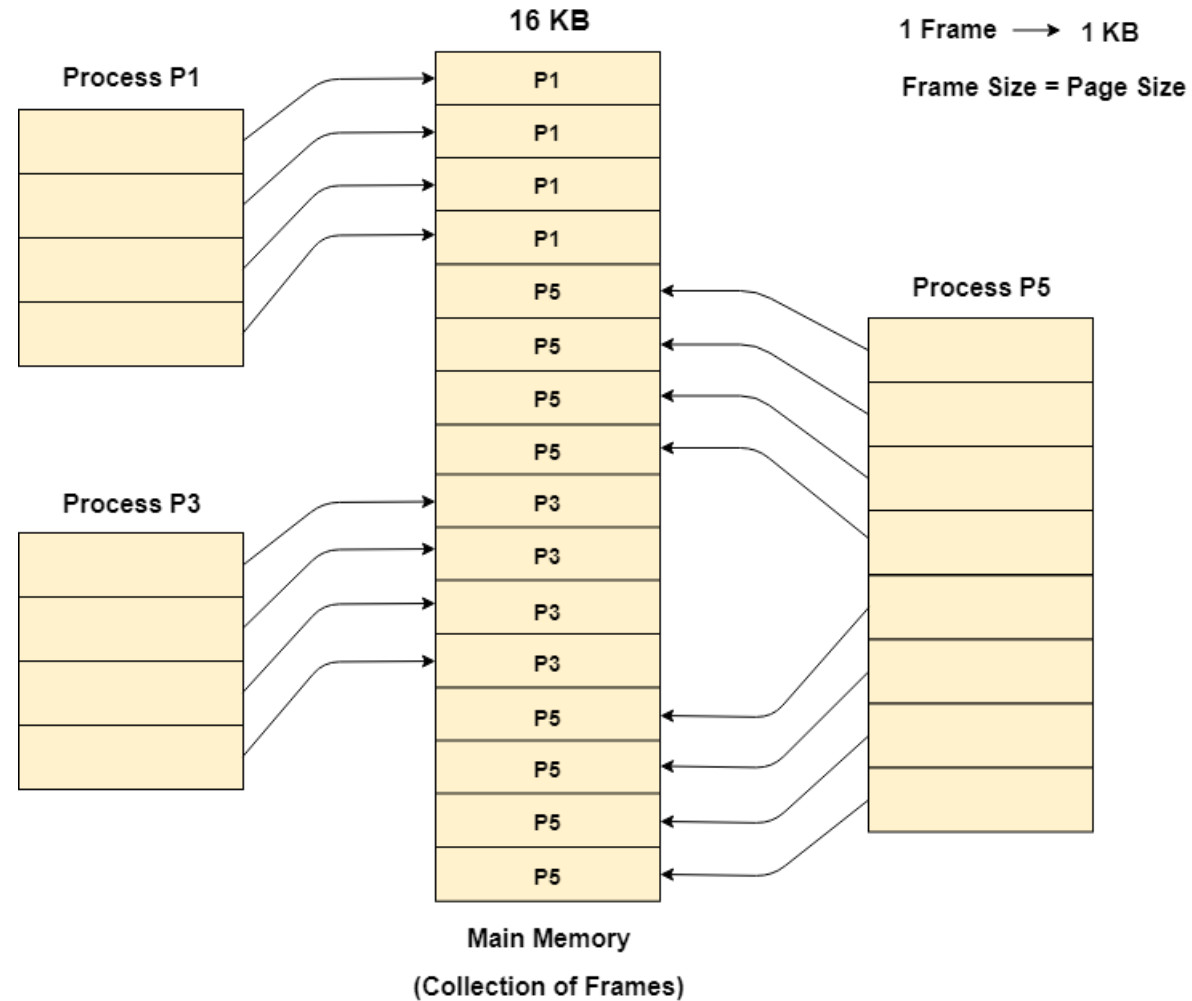
Paging (contd..)



Paging (contd..)

- Let us consider that, P2 and P4 are moved to waiting state after some time. Now, 8 frames become empty and therefore other pages can be loaded in that empty place. The process P5 of size 8 KB (8 pages) is waiting inside the ready queue.
- Given the fact that, we have 8 non contiguous frames available in the memory and paging provides the flexibility of storing the process at the different places. Therefore, we can load the pages of process P5 in the place of P2 and P4.

Paging (contd..)



Paging

Paging (contd..)

Advantages of Paging

- Easy to use memory management algorithm
- No need for external Fragmentation
- Swapping is easy between equal-sized pages and page frames.

Disadvantages of Paging

- May cause Internal fragmentation
- Complex memory management algorithm
- Page tables consume additional memory.
- Multi-level paging may lead to memory reference overhead.

Translating Logical Address into Physical Address

- CPU always generates a logical address.
- A physical address is needed to access the main memory.

Steps to translate logical address into physical address-

Step 1:

CPU generates a logical address consisting of two parts-**Page Number** and **Page Offset**.

- Page Number specifies the specific page of the process from which CPU wants to read the data.
- Page Offset specifies the specific word on the page that CPU wants to read.

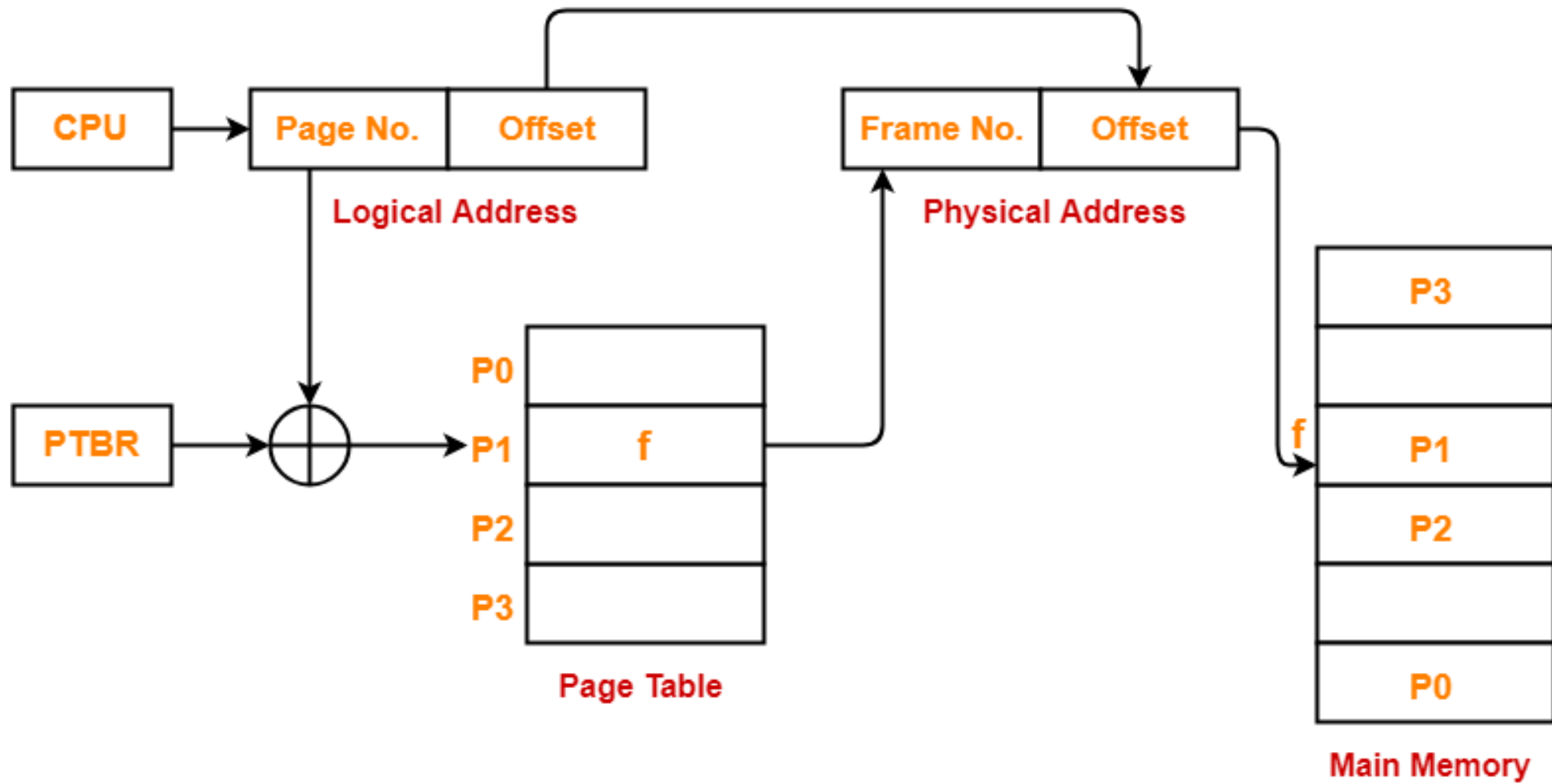
Translating Logical Address into Physical Address (contd..)

Step 2:

For the page number generated by the CPU, Page Table provides the corresponding frame number (base address of the frame) where that page is stored in the main memory.

Step 3:

The frame number combined with the page offset forms the required physical address. Frame number specifies the specific frame where the required page is stored.



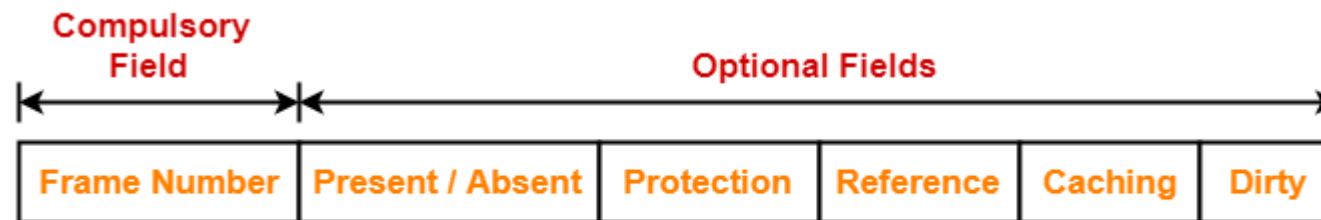
Translating Logical Address into Physical Address

Page Table

- Page table is a data structure.
- It maps the page number referenced by the CPU to the frame number where that page is stored.
- It contains information like: page number, offset, present/absent bit, modified bit etc.
- It is stored in the main memory.
- Number of entries in a page table = Number of pages in which the process is divided.
- Page Table Base Register (PTBR) contains the base address of page table.
- Each process has its own independent page table.

Page Table (contd..)

- A page table entry contains several information about the page. The information contained in the page table entry varies from operating system to operating system.
- The most important information in a page table entry is frame number.
- In general, each entry of a page table contains the following information-



Page Table Entry Format

1. Frame Number-

- Frame number specifies the frame where the page is stored in the main memory.
- The number of bits in frame number depends on the number of frames in the main memory.

2. Present / Absent Bit-

- This bit is also sometimes called as valid / invalid bit.
- This bit specifies whether that page is present in the main memory or not.
- If the page is not present in the main memory, then this bit is set to 0 otherwise set to 1.

Page Table (contd..)

3. Protection Bit-

- This bit is also sometimes called as “Read / Write bit”.
- This bit is concerned with the page protection.
- It specifies the permission to perform read and write operation on the page.
- If only read operation is allowed to be performed and no writing is allowed, then this bit is set to 0.
- If both read and write operation are allowed to be performed, then this bit is set to 1.

4. Reference Bit-

- Reference bit specifies whether that page has been referenced in the last clock cycle or not.
- If the page has been referenced recently, then this bit is set to 1 otherwise set to 0.

Page Table (contd..)

5. Caching Enabled / Disabled-

- This bit enables or disables the caching of page.
- Whenever freshness in the data is required, then caching is disabled using this bit.
- If caching of the page is disabled, then this bit is set to 1 otherwise set to 0.

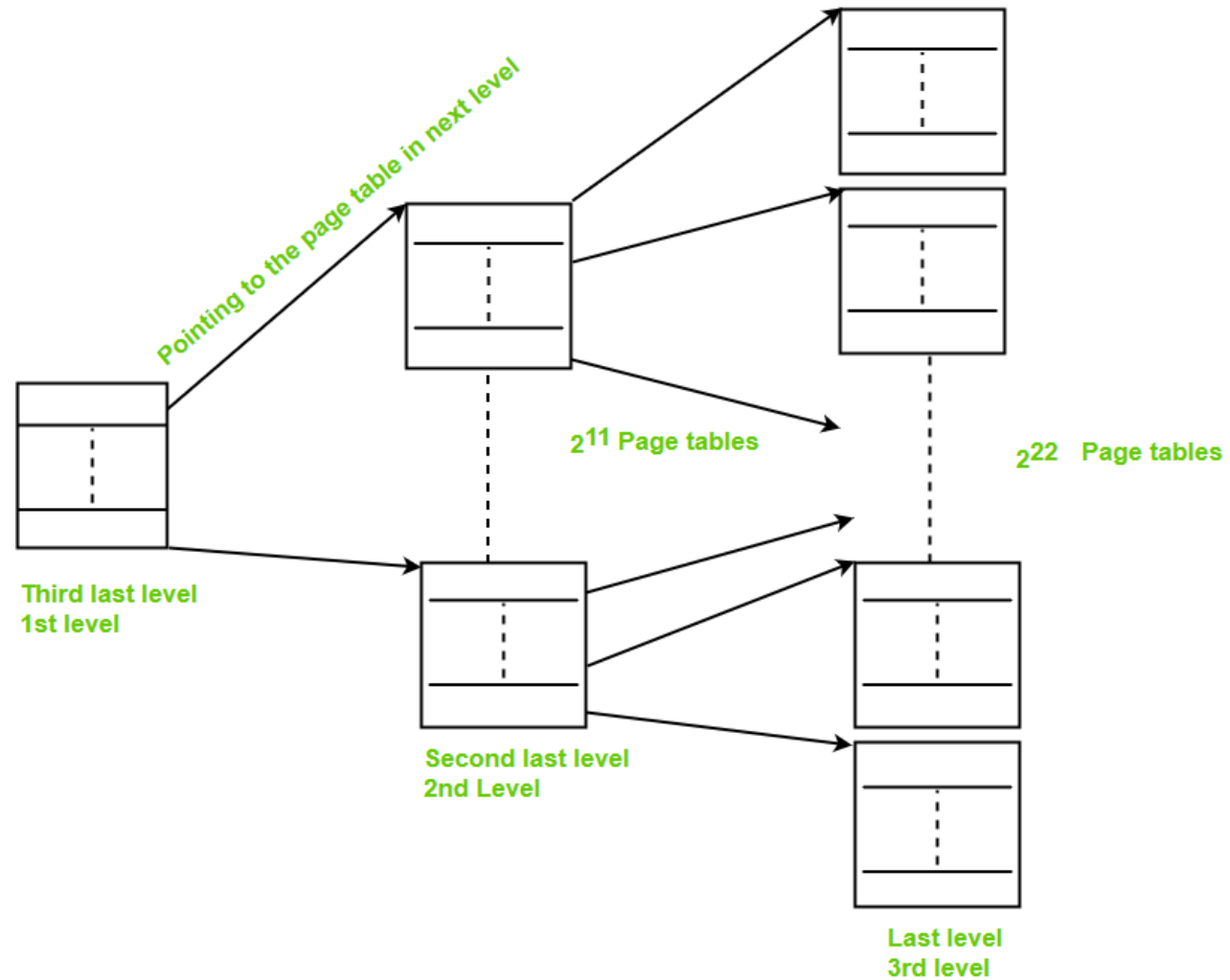
6. Dirty Bit-

- This bit is also sometimes called as “Modified bit”.
- This bit specifies whether that page has been modified or not.
- If the page has been modified, then this bit is set to 1 otherwise set to 0.

Types of page tables

- Hierarchical Page Table
- Inverted Page Table
- Shared Page Table

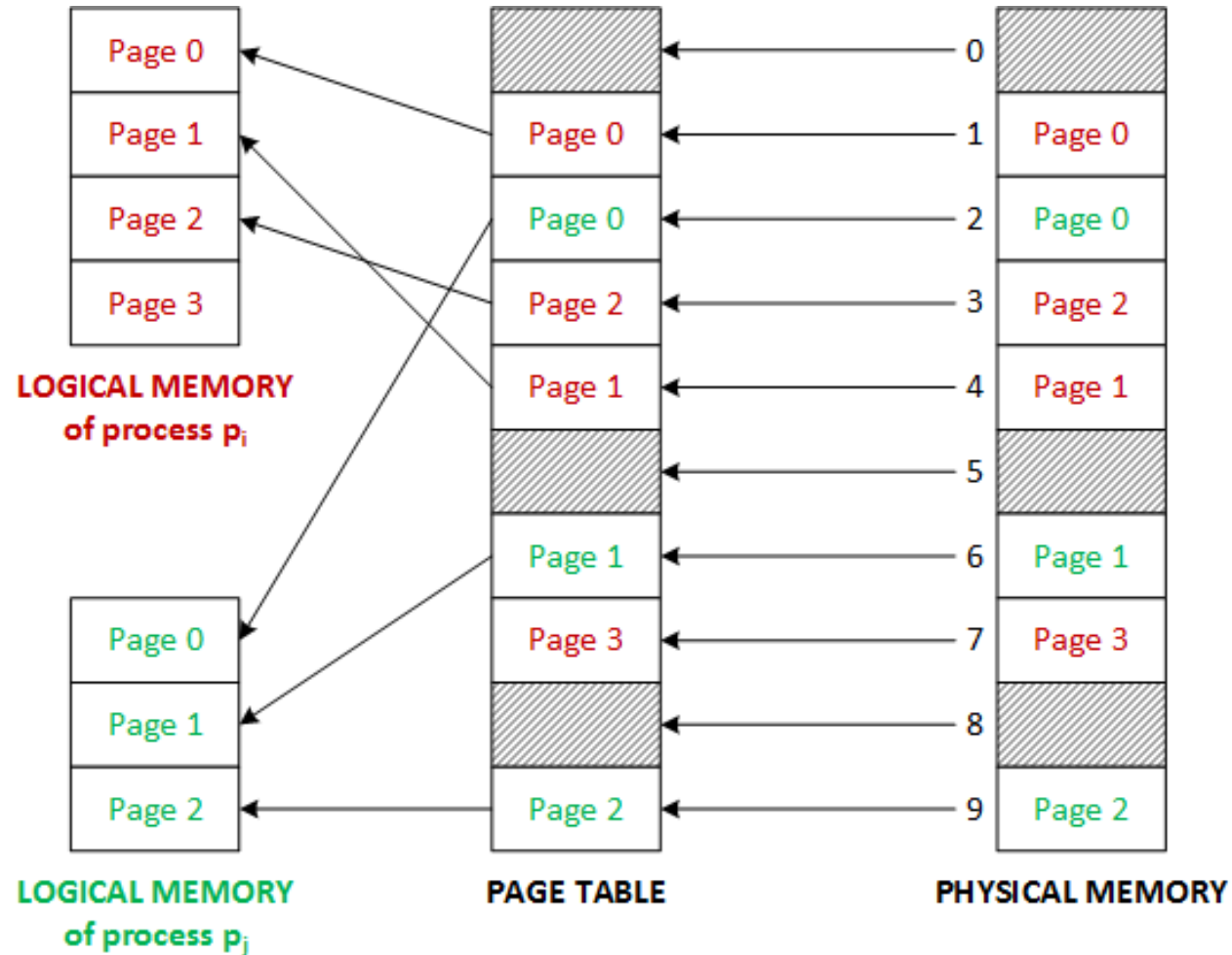
Hierarchical Page Table



Hierarchical Page Table (contd..)

- Hierarchical Page Table (Multilevel Page table) consist of two or more levels of page tables in a hierarchical manner.
- The entries of the level 1 page table are pointers to a level 2 page table and entries of the level 2 page tables are pointers to a level 3 page table and so on. The entries of the last level page table stores actual frame information.
- In multilevel paging whatever may be levels of paging all the page tables will be stored in main memory. So it requires more than one memory access to get the physical address of page frame.

Inverted Page Table



Inverted Page Table (Contd..)

- Inverted Page Table is the global page table which is maintained by the OS for all the processes.
- In inverted page table, the number of entries is equal to the number of frames in the main memory. It can be used to overcome the drawbacks of page table.
- There is always a space reserved for the page regardless of the fact that whether it is present in the main memory or not. However, this is simply the wastage of the memory if the page is not present.
- We can save this wastage by just inverting the page table. We can save the details only for the pages which are present in the main memory.

Inverted Page Table (Contd..)

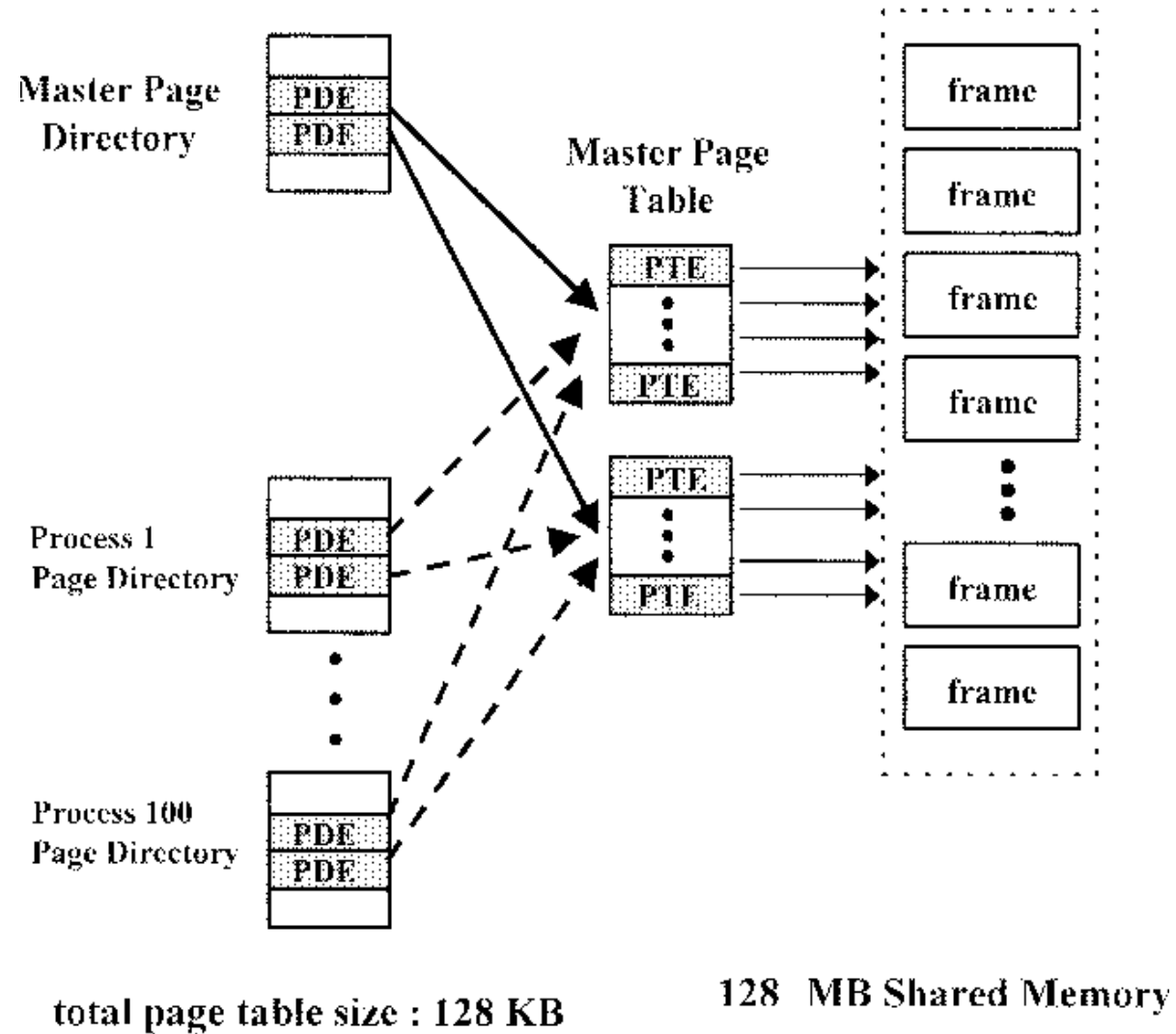
Advantages:

- Reduced memory space

Disadvantages:

- Longer lookup time
- Difficult to implement the shared memory in the page tables

Shared Page Table



Shared Page Table (contd..)

- In this table, the concept is that the pages may be shared by multiple processes.
- It eliminates the redundant page table and significantly reduces the number of minor page faults.
- The primary purpose of sharing page tables is improved performance for large applications that share big memory areas between multiple processes.

Demand Paging

- Demand paging is identical to the paging system with swapping.
- In demand paging, a page is delivered into the memory on demand i.e., only when a reference is made to a location on that page.
- In a system that uses demand paging, the operating system copies a disk page into physical memory only if an attempt is made to access it and that page is not already in memory.

Demand Paging (contd..)

Advantages:

- Large virtual memory
- More efficient use of memory
- There is no limit on degree of multiprogramming

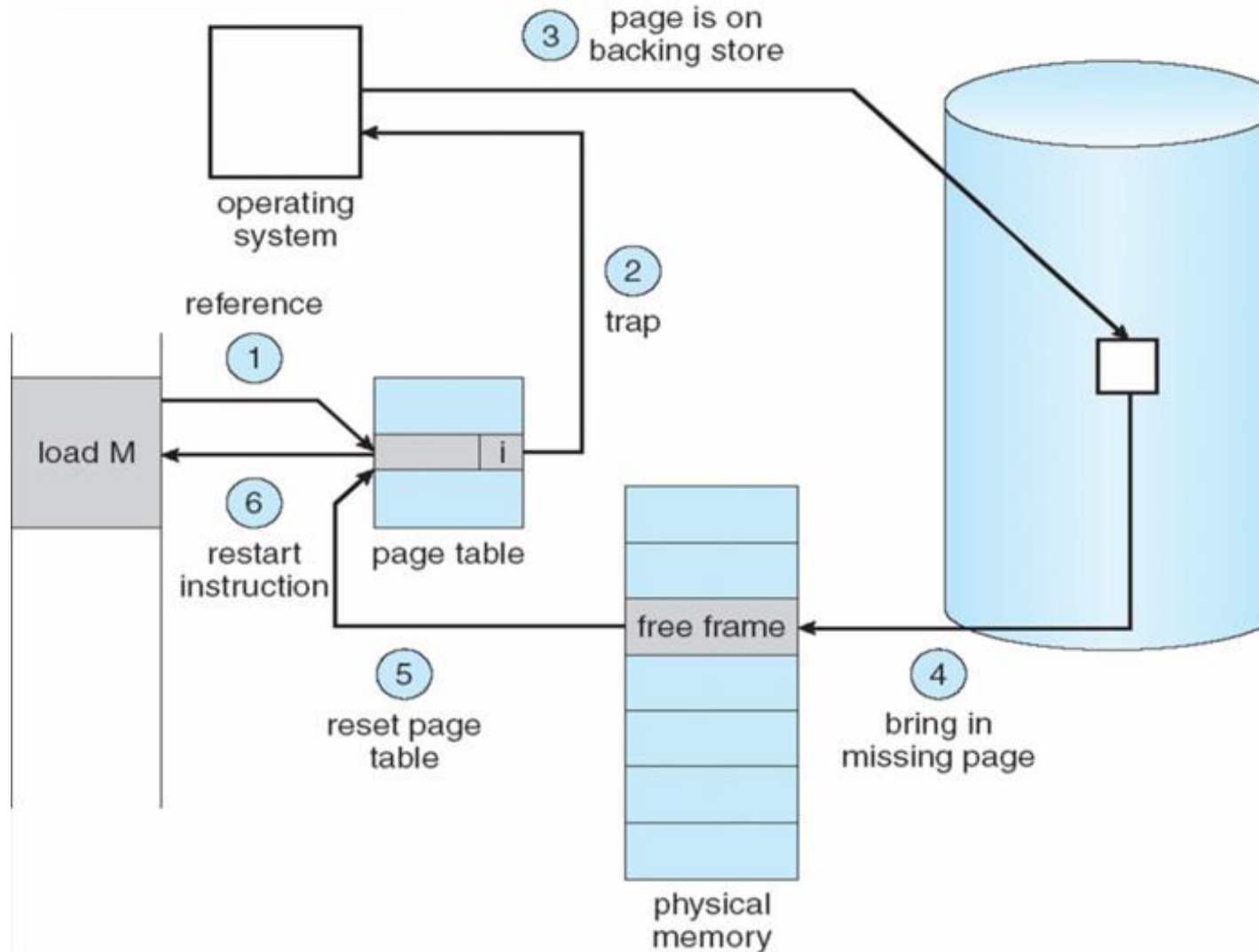
Disadvantages:

- Number of tables and the amount of processor overhead for handling page interrupts are greater than in the case of the simple paged management techniques.

Page Faults

- A page fault occurs when a program attempts to access a block of memory that is not stored in the physical memory, or RAM.
- The fault notifies the operating system that it must locate the data in virtual memory, then transfer it from the storage device, such as an HDD or SSD, to the system RAM.
- Though the term "page fault" sounds like an error, page faults are common and are part of the normal way computers handle virtual memory.
- Most page faults are handled without any problems. However, an invalid page fault may cause a program to hang or crash.

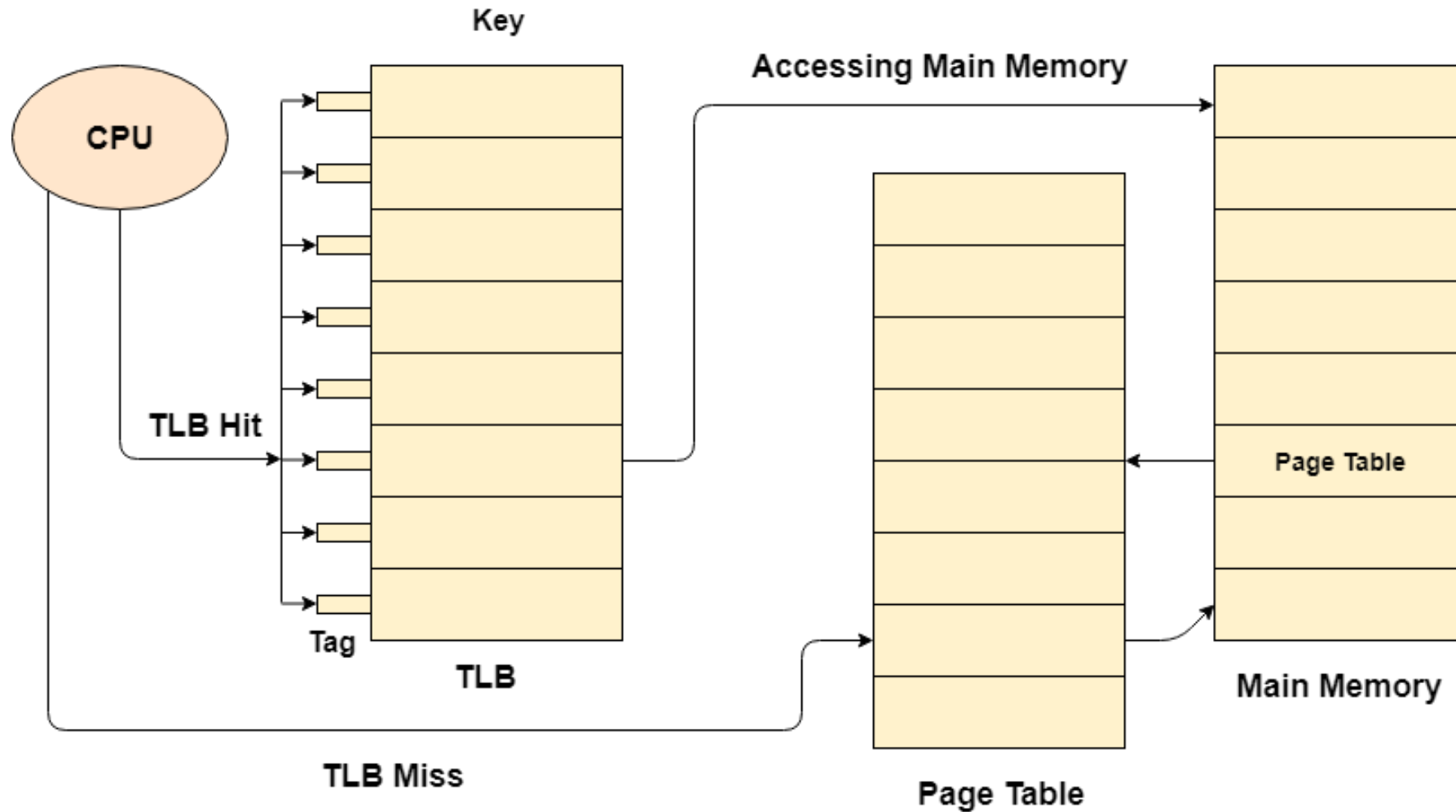
Handling Page faults



Handling Page faults (contd..)

1. Check the location of the referenced page in the Page table.
2. If a page fault occurred, call on the operating system to fix it.
3. Using the frame replacement algorithm, find the frame location.
4. Read the data from disk to memory.
5. Update the page map table for the process.
6. The instruction that caused the page fault is restarted when the process resumes execution.

Translation Look-Aside Buffer (TLB)



Translation Look-Aside Buffer (TLB)

- A Translation look aside buffer can be defined as a memory cache which can be used to reduce the time taken to access the page table again and again.
- It is a memory cache which is closer to the CPU and the time taken by CPU to access TLB is lesser than that taken to access main memory.
- In other words, we can say that TLB is faster and smaller than the main memory but cheaper and bigger than the register.

Translation Look-Aside Buffer (TLB)

- TLB follows the concept of locality of reference which means that it contains only the entries of those many pages that are frequently accessed by the CPU.
- TLB hit is a condition where the desired entry is found in TLB buffer. If this happens then the CPU simply access the actual location in the main memory.
- However, if the entry is not found in TLB (TLB miss) then CPU has to access page table in the main memory and then access the actual frame in the main memory.

Page Replacement Algorithm

- The page replacement algorithm decides which memory page is to be replaced.
- The process of replacement is sometimes called swap out or write to disk.
- Page replacement is done when the requested page is not found in the main memory (page fault).
- Different page replacement algorithms suggest different ways to decide which page to replace. The target for all algorithms is to reduce number of page faults.

Page Replacement Algorithms

Different page replacement algorithms are:

1. FIFO page replacement algorithm
2. Optimal page replacement algorithm
3. LRU page replacement algorithm
4. Second chance page replacement algorithm
5. LFU page replacement algorithm

FIFO page replacement algorithm

- It is the simplest page replacement algorithm.
- In this algorithm, OS keeps track of all pages in the memory in a queue where oldest page is in the front of the queue. When a page needs to be replaced, page in the front of the queue is selected for removal.
- **Advantages:**
 1. Easy to understand and program
 2. Distribute fair chance to all

FIFO page replacement algorithm (contd..)

- **Disadvantages:**

- 1.It is likely to replace heavily (or constantly) used pages and they can be still required for further processing.
- 2.It is not very effective.
- 3.System needs to keep track for each frame.
- 4.Bad replacement choice increases the page fault rate and slow process execution.
- 5.Sometimes it behaves abnormally. This behavior is called Belady's anomaly.

Example 1

- Given references to the following pages by a program: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1. How many page faults will occur if the program has three page frames using FIFO page replacement algorithm?

Request	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
Frame 2		0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
Frame 3			1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
PF	PF	PF	PF	PF	X	PF	PF	PF	PF	PF	PF	X	X	PF	PF	X	X	PF	PF	PF

Number of Page Faults = 15

Example 2

Given references to the following pages by a program: 0, 9, 0, 18, 1, 8, 7, 8, 7, 1, 2, 8, 2, 7, 8, 2, 3, 8, 3. How many page faults will occur if the program has three page frames using FIFO page replacement algorithm?

Soln:

Request	0	9	0	18	1	8	7	8	7	1	2	8	2	7	8	2	3	8	3
Frame 1	0	0	0	0	1	1	1	1	1	1	2	2	2	2	2	2	2	2	2
Frame 2		9	9	9	9	8	8	8	8	8	8	8	8	8	8	8	3	3	3
Frame 3				18	18	18	7	7	7	7	7	7	7	7	7	7	7	8	8
PF	PF	PF	X	PF	PF	PF	PF	X	X	X	PF	X	X	X	X	X	PF	PF	X

The number of page faults=9

Belady's Anomaly

- It is the phenomenon in which increasing the number of page frames results in an increase in the number of page faults for certain memory access patterns.
- It is commonly experienced when using the FIFO page replacement algorithm.

Optimal Page Replacement Algorithm (OPR)

- In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.
- It has the lowest page fault rate among all algorithms.
- **Advantages** – Complexity is less and easy to implement. Assistance needed is low i.e Data Structure used are easy and light.
- **Disadvantages** – OPR is perfect, but not possible in practice as the operating system cannot know future requests. Error handling is tough.

Example 1

- Given references to the following pages by a program: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1. How many page faults will occur if the program has three page frames using OPR page replacement algorithm?

Request	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
Frame 2		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
Frame 3			1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
PF	PF	PF	PF	PF	X	PF	X	PF	X	X	PF	X	X	PF	X	X	X	PF	X	X

Number of Page Faults = 9

Example 2

Given references to the following pages by a program: 0, 9, 0, 18, 1, 8, 7, 8, 7, 1, 2, 8, 2, 7, 8, 2, 3, 8, 3. How many page faults will occur if the program has three page frames using OPR page replacement algorithm?

Soln:

Request	0	9	0	18	1	8	7	8	7	1	2	8	2	7	8	2	3	8	3
Frame 1	0	0	0	0	1	1	1	1	1	1	2	2	2	2	2	2	3	3	3
Frame 2		9	9	9	9	8	8	8	8	8	8	8	8	8	8	8	8	8	8
Frame 3				18	18	18	7	7	7	7	7	7	7	7	7	7	7	7	7
PF	PF	PF	X	PF	PF	PF	PF	X	X	X	PF	X	X	X	X	X	PF	X	X

The number of page faults=8

LRU Page Replacement Algorithm

- LRU = Least Recently Used
- In this algorithm, the page that has not been used for longest period of time has to be replaced.
- This strategy is the optimal page replacement algorithm looking backward in time, rather than forward.
- **Advantages** – It is open for full analysis. It is free from Belady's Anomaly.
- **Disadvantages** – It requires additional Data Structure to be implemented. Hardware assistance is high.

Example 1

- Given references to the following pages by a program: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1. How many page faults will occur if the program has three page frames using LRU page replacement algorithm?

Request	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
Frame 2		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
Frame 3			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
PF	PF	PF	PF	PF	X	PF	X	PF	PF	PF	PF	X	X	PF	X	PF	X	PF	X	X

Number of Page Faults = 12

Example 2

Given references to the following pages by a program: 0, 9, 0, 18, 1, 8, 7, 8, 7, 1, 2, 8, 2, 7, 8, 2, 3, 8, 3. How many page faults will occur if the program has three page frames using LRU page replacement algorithm?

Soln:

Request	0	9	0	18	1	8	7	8	7	1	2	8	2	7	8	2	3	8	3
Frame 1	0	0	0	0	0	8	8	8	8	8	2	2	2	2	2	2	2	2	2
Frame 2		9	9	9	1	1	1	1	1	1	1	1	1	7	7	7	3	3	3
Frame 3				18	18	18	7	7	7	7	7	8	8	8	8	8	8	8	8
PF	PF	PF	X	PF	PF	PF	PF	X	X	X	PF	PF	X	PF	X	X	PF	X	X

The number of page faults=10

Second Chance Page Replacement Algorithm

- It is the modified form of the FIFO page replacement algorithm.
- The reference bit is used to decide which page to replace. The page is replaced if the reference bit is 0. However, the page is given second chance if the reference bit is 1 and the next page in FIFO is replaced.
- When a page gets a second chance, its reference bit is cleared and its arrival time is reset to the current time.
- One way to implement the second chance algorithm is by using circular queue.

Second Chance Page Replacement Algorithm

Advantages:

- Improvement over FIFO.
- Allows commonly used pages to stay in queue.

Disadvantages:

- Still suffers from Belady's anomaly

Example 1

- Given references to the following pages by a program: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1. How many page faults will occur if the program has three page frames using Second chance page replacement algorithm?

Request	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7 (0)	7 (0)	7 (0)	2 (0)	2 (0)	2 (0)	2 (0)	4 (0)	4 (0)	4 (0)	0 (0)	0 (0)	0 (0)	1 (0)	1 (0)	1 (0)	1 (1)	1 (0)	1 (0)	1 (1)
Frame 2		0 (0)	0 (0)	0 (0)	0 (1)	0 (0)	0 (1)	0 (0)	0 (0)	3 (0)	3 (0)	3 (1)	3 (1)	3 (0)	3 (0)	0 (0)	0 (0)	0 (0)	0 (1)	0 (1)
Frame 3			1 (0)	1 (0)	1 (0)	3 (0)	3 (0)	3 (0)	2 (0)	2 (0)	2 (0)	2 (0)	2 (1)	2 (0)	2 (1)	2 (0)	2 (0)	7 (0)	7 (0)	7 (0)
PF	PF	PF	PF	PF	X	PF	X	PF	PF	PF	PF	X	X	PF	X	PF	X	PF	X	X

Number of Page Faults = 12

Example 2

Given references to the following pages by a program: 0, 9, 0, 18, 1, 8, 7, 8, 7, 1, 2, 8, 2, 7, 8, 2, 3, 8, 3. How many page faults will occur if the program has three page frames using Second Chance page replacement algorithm?

Soln:

Request	0	9	0	18	1	8	7	8	7	1	2	8	2	7	8	2	3	8	3
Frame 1																			
Frame 2																			
Frame 3																			
PF																			

The number of page faults=

LFU Page Replacement Algorithm

- LFU = Least Frequently Used.
- The frequency of the page is taken into consideration for selecting the page to replace.
- In this algorithm, the page with the minimum frequency is selected for replacement.
- If all the pages have same minimum frequency, the page that has arrived first is the one to be replaced (similar to FIFO).

Example 1

- Given references to the following pages by a program: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1. How many page faults will occur if the program has three page frames using LFU page replacement algorithm?

Request	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	3	3	3	3	3	3	3	3	3	3	3
Frame 2		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Frame 3			1	1	1	3	3	3	2	2	2	2	2	1	2	2	1	7	7	7
PF	PF	PF	PF	PF	X	PF	X	PF	PF	PF	X	X	X	PF	PF	X	PF	PF	X	PF

Number of Page Faults = 13

Example 2

Given references to the following pages by a program: 0, 9, 0, 18, 1, 8, 7, 8, 7, 1, 2, 8, 2, 7, 8, 2, 3, 8, 3. How many page faults will occur if the program has three page frames using LRU page replacement algorithm?

Soln:

Request	0	9	0	18	1	8	7	8	7	1	2	8	2	7	8	2	3	8	3
Frame 1	0	0	0	0	0	0	0	0	0	1	2	2	2	2	2	2	2	2	2
Frame 2		9	9	9	1	1	7	7	7	7	7	7	7	7	7	7	3	3	3
Frame 3				18	18	8	8	8	8	8	8	8	8	8	8	8	8	8	8
PF	PF	PF	X	PF	PF	PF	PF	X	X	PF	PF	X	X	X	X	X	PF	X	X

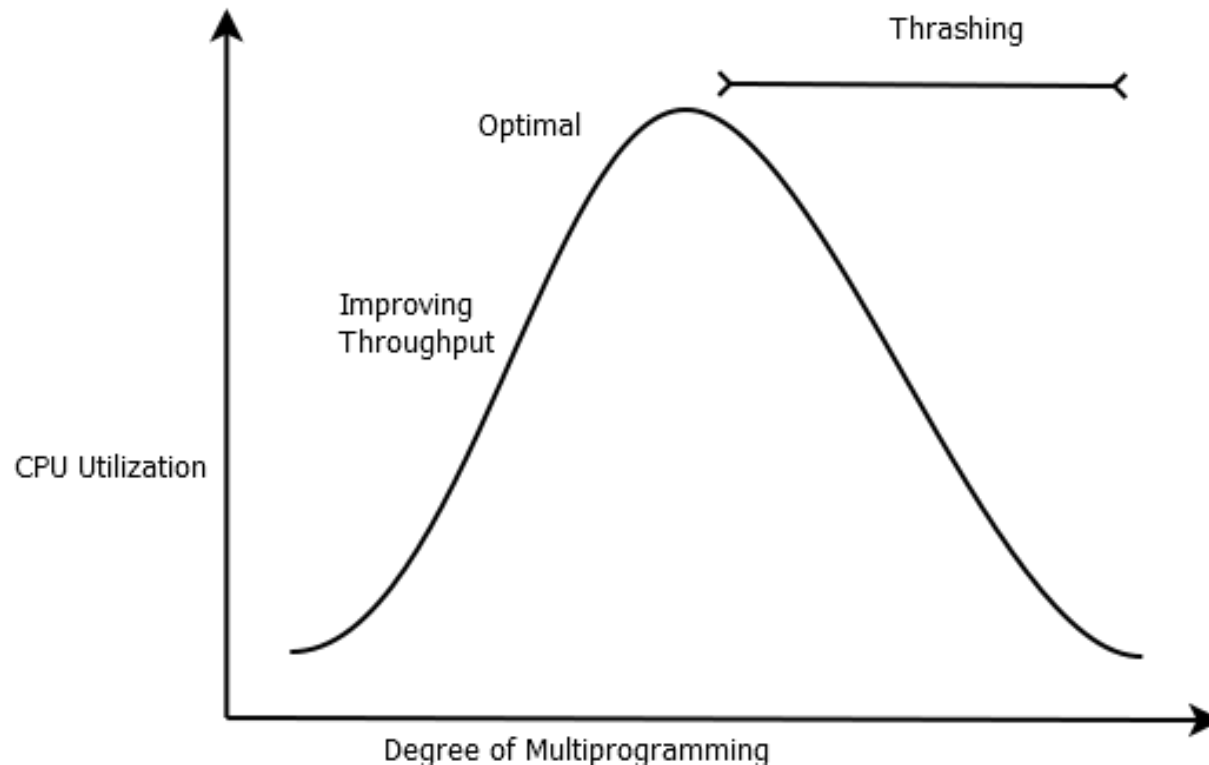
The number of page faults=9

Thrashing

- It is the condition in which system is spending a major portion of its time in servicing the page faults as compared to the major processing.
- If a process is spending more time for paging than executing, then it is said to be thrashing.
- If the process doesn't have enough frames to hold all the pages for its execution, OS swaps pages in and out very frequently to keep executing. Sometimes, the pages which will be required in the near future have to be swapped out.
- The effect of Thrashing is: performance degradation, more CPU time is wasted for swapping process.

Thrashing (contd..)

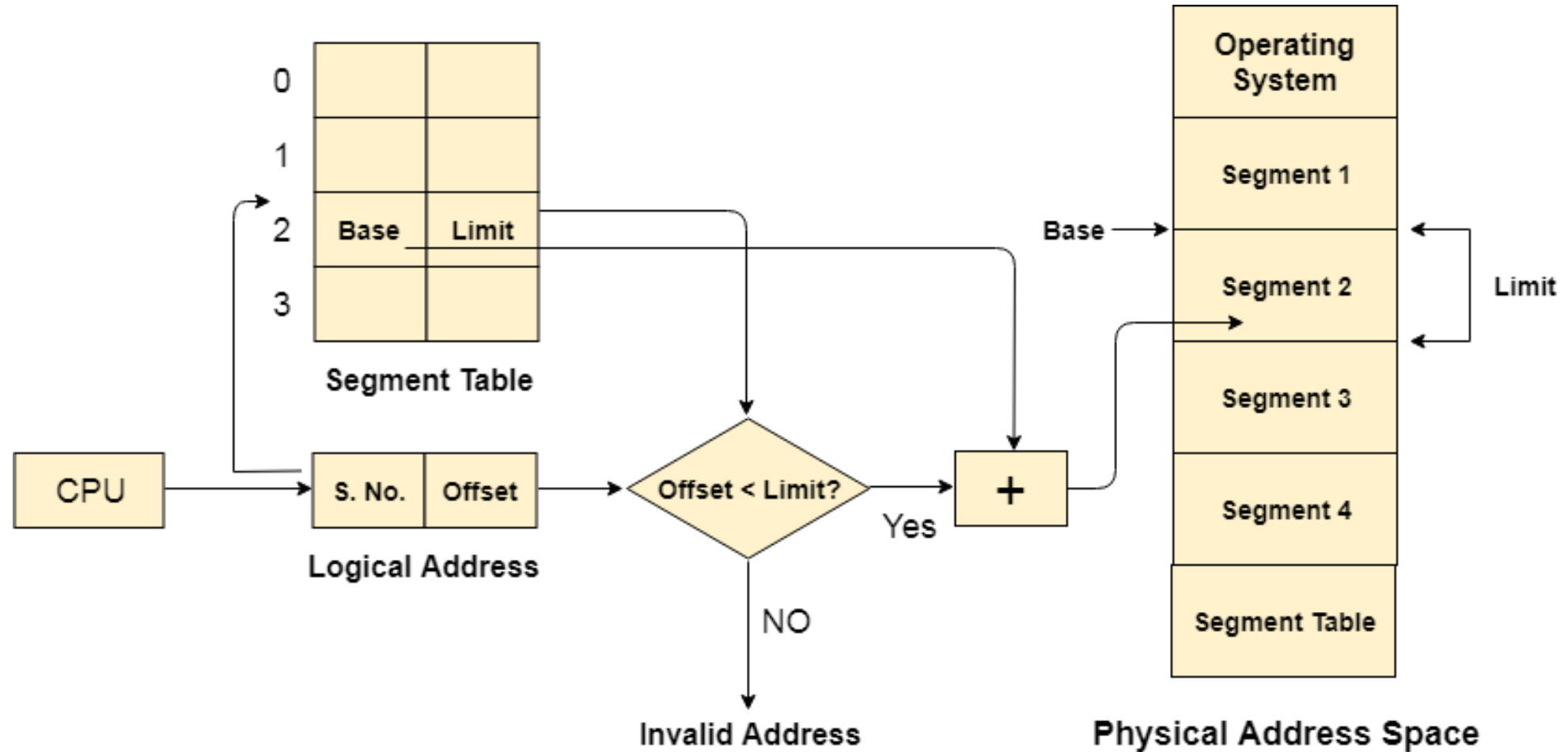
- To prevent thrashing, we must provide processes with as many frames as they really need.



Segmentation

- Segmentation is a memory management technique in which, the memory is divided into the variable size parts. Each part is known as segment which can be allocated to a process.
- The details about each segment are stored in a table called as segment table. Segment table is stored in one (or many) of the segments.
- Segment table contains mainly two information about segment:
 - Base: It is the base address of the segment
 - Limit: It is the length of the segment.

Translation of Logical address into physical address by segment table



Translation of Logical address into physical address by segment table

- CPU generates a logical address which contains two parts: Segment Number and Offset.
- The Segment number is mapped to the segment table. The limit of the respective segment is compared with the offset. If the offset is less than the limit then the address is valid otherwise it throws an error as the address is invalid.
- In the case of valid address, the base address of the segment is added to the offset to get the physical address of actual word in the main memory.

Advantages and Disadvantages of Segmentation

Advantages of Segmentation

- No internal fragmentation
- Average Segment Size is larger than the actual page size.
- Less overhead
- It is easier to relocate segments than entire address space.
- The segment table is of lesser size as compare to the page table in paging.

Disadvantages of Segmentation

- It can have external fragmentation.
- it is difficult to allocate contiguous memory to variable sized partition.
- Costly memory management algorithms.

Segmentation (Contd..)

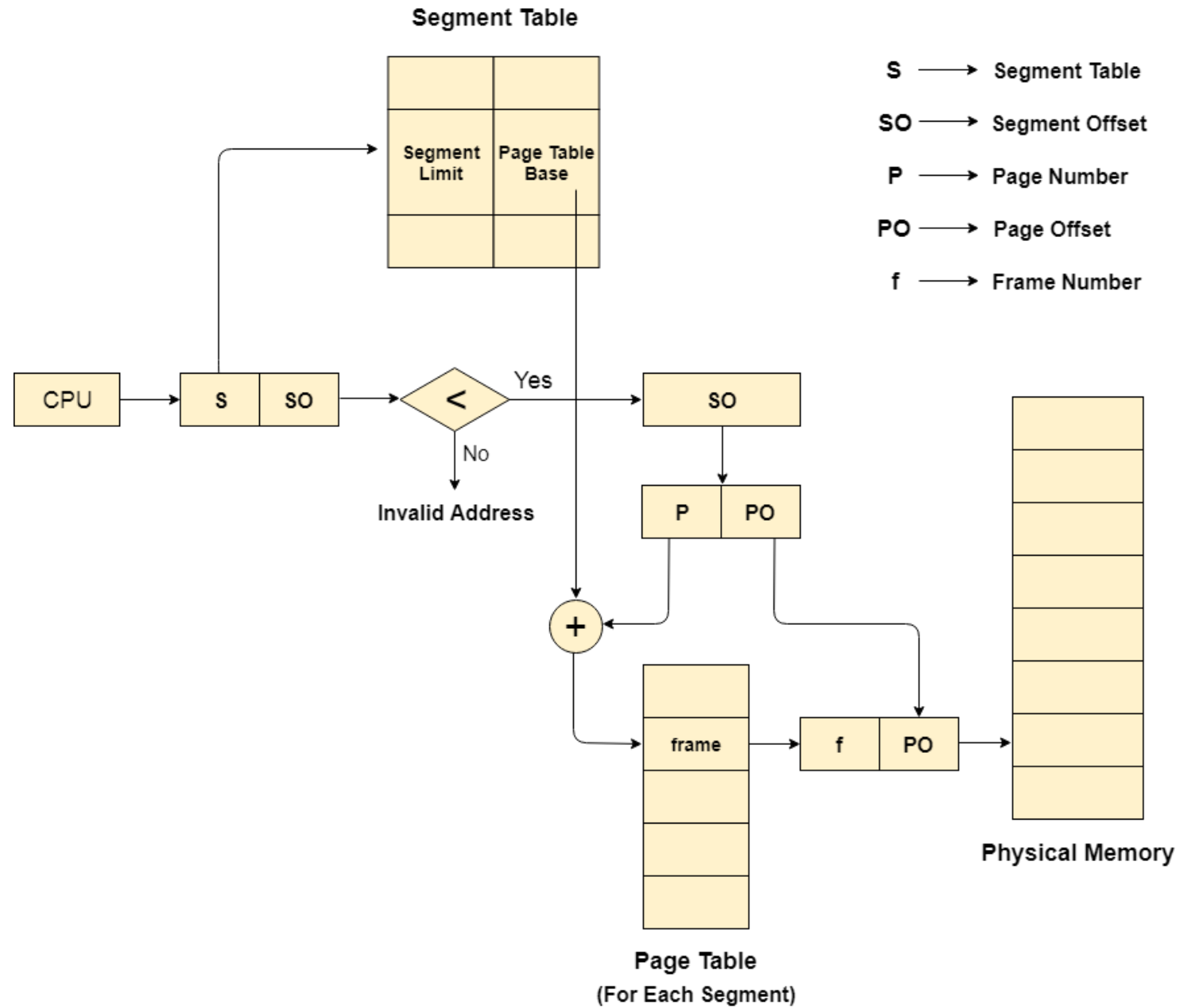
- Paging divides all the process into the form of pages regardless of the fact that a process can have some relative parts of functions which needs to be loaded in the same page. OS doesn't care about the User's view of the process. It may divide the same function into different pages and those pages may or may not be loaded at the same time into the memory. It decreases the efficiency of the system.
- It is better to have segmentation which divides the process into the segments. Each segment contain same type of functions such as main function can be included in one segment and the library functions can be included in the other segment.

Paging Vs Segmentation

Sr No.	Paging	Segmentation
1	Non-Contiguous memory allocation	Non-contiguous memory allocation
2	Paging divides program into fixed size pages.	Segmentation divides program into variable size segments.
3	OS is responsible	Compiler is responsible.
4	Paging is faster than segmentation	Segmentation is slower than paging
5	Paging is closer to Operating System	Segmentation is closer to User
6	It suffers from internal fragmentation	It suffers from external fragmentation
7	There is no external fragmentation	There is no external fragmentation
8	Logical address is divided into page number and page offset	Logical address is divided into segment number and segment offset
9	Page table is used to maintain the page information.	Segment Table maintains the segment information
10	Page table entry has the frame number and some flag bits to represent details about pages.	Segment table entry has the base address of the segment and some protection bits for the segments.

Segmentation with Paging (Multics)

- Segmented paging
- Pure segmentation is not very popular and not being used in many of the operating systems. However, Segmentation can be combined with Paging to get the best features out of both the techniques.
- In Segmented Paging, the main memory is divided into variable size segments which are further divided into fixed size pages.
- Each Segment has a page table which means every program has multiple page tables.
- The logical address is represented as Segment Number (base address), Page number and page offset.
 - Segment Number → It points to the appropriate Segment Number.
 - Page Number → It Points to the exact page within the segment
 - Page Offset → Used as an offset within the page frame
- Each Page table contains the various information about every page of the segment. The Segment Table contains the information about every segment. Each segment table entry points to a page table entry and every page table entry is mapped to one of the page within a segment.



Advantages and Disadvantages of Segmented Paging

Advantages

- It reduces memory usage.
- Page table size is limited by the segment size.
- Segment table has only one entry corresponding to one actual segment.
- External Fragmentation is not there.
- It simplifies memory allocation.

Disadvantages

- Internal Fragmentation will be there.
- The complexity level will be much higher as compare to paging.
- Page Tables need to be contiguously stored in the memory.